COS40007 Artificial Intelligence Engineering

Portfolio Assessment 3

Advanced ML – Classification using Machine Learning and Deep Learning

Option 3c: LSTM for Classification

Dataset: Appliances Energy Prediction (UCI Machine Learning Repository)

Student name: Dinh Phu Nguyen

Student ID: 105027282

Tutor: Dr. Duc Tho Le

Unit: COS40007 Artificial Intelligence Engineering

Task: Portfolio Assessment 3

Submission date: 11/1/2026

GitHub repository: https://github.com/phu1009/COS40007---AI-Engineering-

Acknowledgement of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

We are honoured to recognise our connection to Wurundjeri Country, history, culture and spirituality through these locations, and strive to ensure that we operate in a manner that respects and honours the Elders and Ancestors of these lands.

We also respectfully acknowledge Swinburne's Aboriginal and Torres Strait Islander staff, students, alumni, partners and visitors.

We also acknowledge and respect the Traditional Owners of lands across Australia, their Elders, Ancestors, cultures and heritage, and recognise the continuing sovereignties of all Aboriginal and Torres Strait Islander Nations.

Advanced ML and Deep Learning Report

1. Introduction and Problem Definition

This report continues Portfolio Assessments 1 and 2 on the **Appliances Energy Prediction** dataset: 19,735 readings taken every 10 minutes (11 January – 27 May 2016) of the appliance energy use of a low-energy house, with indoor temperature and humidity sensors and outdoor weather data (Candanedo et al., 2017). The target is the consumption class defined in PA2: **Low** (≤ 50 Wh per 10 minutes), **Medium** (60–80 Wh) and **High** (≥ 90 Wh). The data is a real time series and the target is categorical, so **option 3c – LSTM for Classification** was chosen.

To compare all models fairly, the dataset was kept fixed for Parts A and B (Table 1).

Table 1: Design decisions kept fixed for Parts A and B.

Decision	Choice	Reason
Cleaning	PA1: date → datetime; rv1, rv2, RH_6 removed	noise columns and a faulty sensor (PA1)
Features	24 cleaned predictors + hour_sin, hour_cos (26)	sensor readings plus the daily routine; temporal patterns are left for the LSTM to learn
Split	chronological 70/30: train 11 Jan – 16 Apr, test 16 Apr – 27 May	shuffled splits are optimistic for time series (PA2 §10.2); LSTM windows overlap
Test set	the same 5,921 readings for every model	fair comparison in Part C

2. Part A – Traditional Machine Learning Models

X (26 features) and y (class 0/1/2) were separated and split chronologically (13,814 training and 5,921 test readings). A StandardScaler was fitted on the training set only. Figure 1 shows the class distribution and the class mix of the two periods.

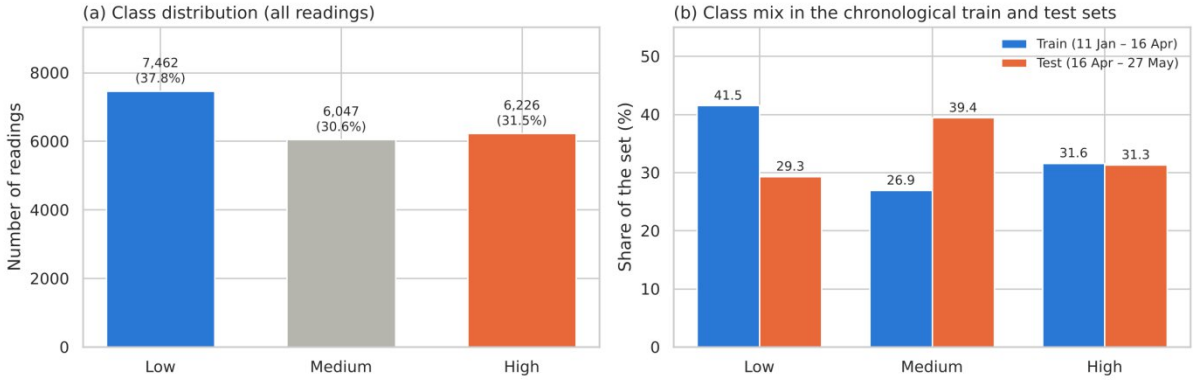


Figure 1: Class distribution (a) and class mix of the chronological train and test sets (b).

Two models were trained: a **Random Forest** (300 trees, min_samples_leaf = 5) and an **MLP Classifier** (hidden layers 100 and 50, ReLU, Adam, alpha = 0.001, early stopping), together with a majority-class baseline. Cross-validation used a 5-fold **TimeSeriesSplit** on the training period, so that each fold validates on data later than its training data. Both models were saved as .pkl files.

Table 2: Part A comparison on the test period (best in bold).

Model	Test accuracy	Weighted F1	Test log loss	CV accuracy (TimeSeriesSplit)	Train accuracy
Majority baseline	0.2930	0.1328	25.48	0.3850 ± 0.0547	0.4146

Model	Test accuracy	Weighted F1	Test log loss	CV accuracy (TimeSeriesSplit)	Train accuracy
Random Forest	**0.5128**	**0.5055**	**0.9174**	**0.5903 ± 0.0371**	0.8956
MLP Classifier	0.4849	0.4860	1.4593	0.5793 ± 0.0618	0.8016

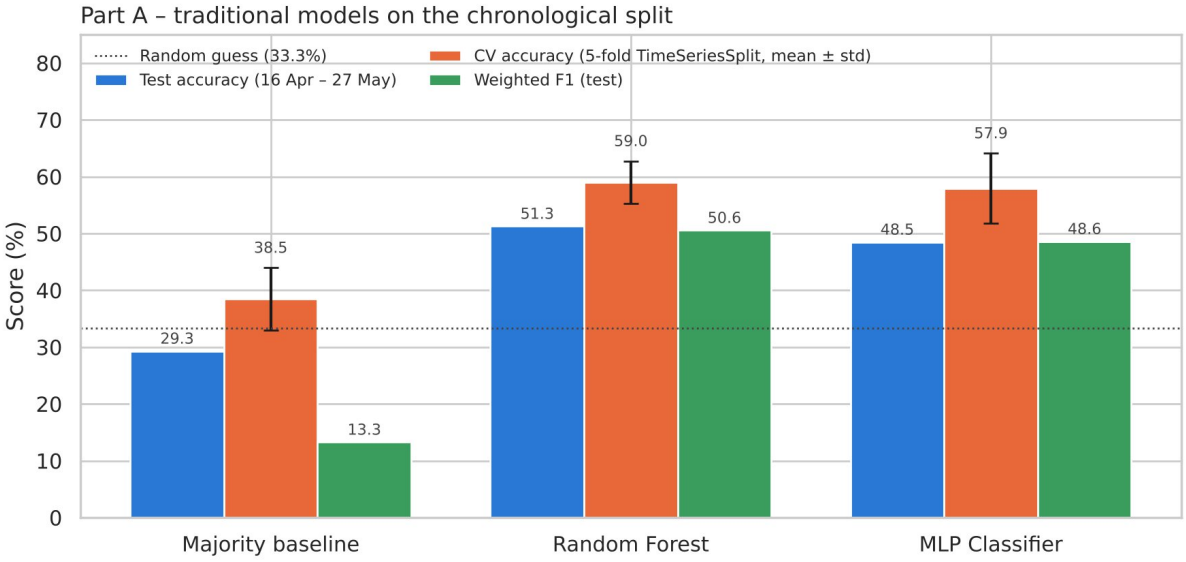


Figure 2: Part A: test accuracy, CV accuracy and weighted F1 of the traditional models.

- **Random Forest is the best traditional model:** test accuracy **51.28%**, weighted F1 **0.506** and log loss **0.917**, compared with 48.49%, 0.486 and 1.459 for the MLP. It is also better in the time-series cross-validation (59.0% ± 3.7 vs 57.9% ± 6.2) and varies less between folds.
- **Why the Random Forest wins:**
  - It averages 300 de-correlated trees, which cancels much of the noise of a single tree (PA2 trees reached 100% training accuracy).
  - It handles skewed sensors and interactions such as *time of day* × *lights* without any tuning.
  - Its most important features are hour_sin (0.129) and hour_cos (0.096), together 22% of the importance, followed by lights (0.061) and the living-room and bedroom sensors T8, T2 and RH_8. This matches the daily routine found in PA1 §3.3.
- **The MLP is over-confident:** its log loss (1.459) is much higher than that of the Random Forest, so its wrong predictions are made with high probability. Its internal early-stopping score was **76.9%**, because scikit-learn validates on a *random* 10% of the training rows: their 10-minute neighbours are in the training data. On the later test period it reaches only 48.5%, which shows again (as in PA2 §10.2) that shuffled validation is optimistic for time series.
- **Cross-validation vs test:** the CV scores (58–59%) are higher than the test scores (48–51%). Each CV fold validates on the block right after its training data, while the test period is later in spring and has a different class mix.
- **Overfitting:** training accuracy is 89.6% for the Random Forest and 80.2% for the MLP, far above the test accuracy.
- **Errors by class:** the Random Forest recognises High best (recall 0.69) and Low worst (0.36); many spring Low readings are predicted as Medium.

3. Part B – LSTM for Classification

3.1 Sequence preparation

The 26 features were scaled to 0–1 with a **MinMaxScaler** fitted on the training period. The sequence length was chosen from the memory of the data (Figure 3 and Table 3).

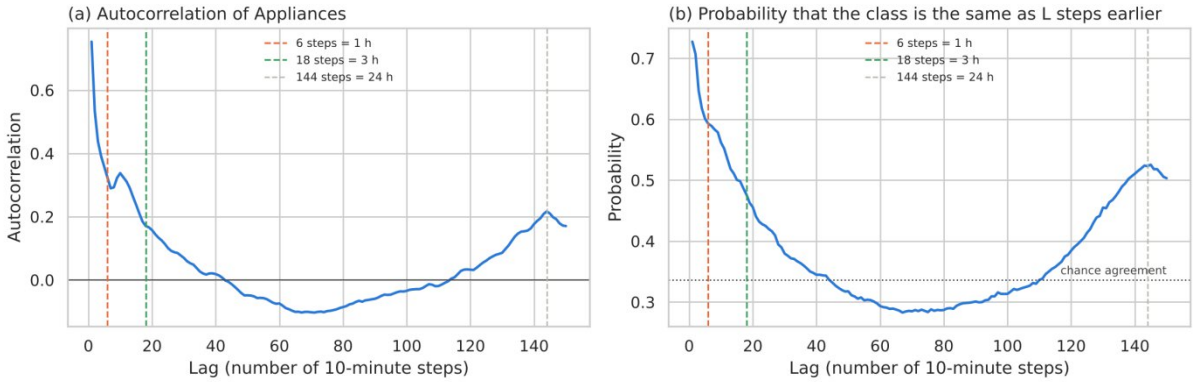


Figure 3: Autocorrelation of Appliances (a) and probability that the class is unchanged after L steps (b).

Table 3: Memory of the time series at different lags.

Lag	Time	Autocorrelation	P(same class)
1	10 min	0.753	0.727
3	30 min	0.438	0.647
6	**1 h (LSTM-1)**	0.324	0.593
12	2 h	0.311	0.535
18	**3 h (LSTM-2)**	0.171	0.476
36	6 h	0.021	0.360
144	24 h	0.217	0.523

Justification of the sequence length.

- **Autocorrelation of Appliances (figure a):** 0.75 at 10 minutes, 0.44 at 30 minutes, 0.32 at 1 hour and 0.17 at 3 hours, then about 0 at 6 hours. It rises again to 0.22 at 24 hours because of the daily cycle, which is already given to every model through hour_sin and hour_cos.
- **Class persistence (figure b):** the class is the same as 1 hour earlier in 59% of the readings and as 3 hours earlier in 48%. At 6 hours this falls to 36%, which is the chance level (0.34).
- **Run lengths:** a High period lasts **69 minutes** on average, a Low period 39 minutes and a Medium period 23 minutes.
- **Conclusion:** the recent past is informative for about 1–3 hours, so two lengths are compared:
    - **LSTM-1 uses 6 steps (1 hour).** This covers a typical High event and the strongest part of the autocorrelation.
    - **LSTM-2 uses 18 steps (3 hours).** This goes up to the point where the autocorrelation has faded (0.17).
    - Longer windows (36+ steps) would mostly add uninformative history and multiply the training time.

Final shapes: training (13,809, 6, 26) and (13,797, 18, 26); test (5,921, 6, 26) and (5,921, 18, 26), i.e. (samples, timesteps, features). The test windows end on the same 5,921 readings as the Part A test set.

Note on normalisation: the test values lie between −0.48 and 1.62, outside the 0–1 training range (for example, spring outdoor temperatures are higher than any winter value). The LSTM has to extrapolate, which is part of the seasonal-shift problem.

Input–output pairs. For every reading t, the input is the window of readings t − L + 1 … t and the output is the class of reading t. Windows that end in the training period form the training set and windows that end in the test period form the test set. Test windows near 16 April use the last training-period readings as inputs only, so the LSTM is evaluated on exactly the same 5,921 readings as the Part A models.

3.2 Architecture

Table 4: LSTM architecture (Adam, learning rate 0.001, sparse categorical crossentropy, accuracy).

Layer	Setting	Output shape (LSTM-1)	Parameters
LSTM	64 units, return_sequences=True	(None, 6, 64)	23,296
Dropout	0.2	(None, 6, 64)	0
LSTM	32 units, return_sequences=False	(None, 32)	12,416
Dropout	0.2	(None, 32)	0
Dense	16 units, ReLU	(None, 16)	528
Dense (output)	3 units, Softmax	(None, 3)	51

```
Model: "LSTM_1_seq6"
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape                   ┃                        Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┩
│ lstm (LSTM)                     │ (None, 6, 64)                  │                        23,296 │
├─────────────────────────────────┼────────────────────────────────┼────────────────────────────────┤
│ dropout (Dropout)               │ (None, 6, 64)                  │                             0 │
├─────────────────────────────────┼────────────────────────────────┼────────────────────────────────┤
│ lstm_1 (LSTM)                   │ (None, 32)                     │                        12,416 │
├─────────────────────────────────┼────────────────────────────────┼────────────────────────────────┤
│ dropout_1 (Dropout)             │ (None, 32)                     │                             0 │
├─────────────────────────────────┼────────────────────────────────┼────────────────────────────────┤
│ dense (Dense)                   │ (None, 16)                     │                           528 │
├─────────────────────────────────┼────────────────────────────────┼────────────────────────────────┤
│ dense_1 (Dense)                 │ (None, 3)                      │                            51 │
└─────────────────────────────────┴────────────────────────────────┴────────────────────────────────┘
Total params: 36,291 (141.76 KB)
Trainable params: 36,291 (141.76 KB)
Non-trainable params: 0 (0.00 B)
```

Figure 4: Screenshot of model.summary() for LSTM-1.

- **Parameters:** 36,291 trainable parameters.
  - First LSTM: 4 gates × (64 × (26 inputs + 64 recurrent) + 64 biases) = 23,296.
  - Second LSTM: 4 × (32 × (64 + 32) + 32) = 12,416.
  - Dense layers: 528 + 51.
- **Sequence length does not change the parameter count:** the same LSTM weights are applied at every time step. LSTM-1 and LSTM-2 therefore have the same 36,291 parameters; a longer window only costs more computation per sample.

- **Shapes:** return_sequences=True makes the first LSTM pass its full output (6 × 64 for LSTM-1) to the second LSTM, which returns only its final state (32 values) to the dense layers.
- **Overfitting risk:** there are about 2.6 parameters per training window (36,291 vs 13,809), so overfitting is a real risk. This is why dropout and early stopping are used.

3.3 Training

Training used a validation split of 20% (the last, most recent 20% of the training windows), **EarlyStopping** (val_loss, patience 10, restore_best_weights) and **ReduceLROnPlateau** (val_loss, factor 0.5, patience 5), with up to 50 epochs. A first run of 10 epochs without callbacks checked that the model learns.

10-epoch check:

- **The model learns:** training loss falls steadily (0.872 → 0.668) and training accuracy rises (59.4% → 70.1%).
- **Validation does not follow:** validation loss is lowest at epoch 4 (0.852) and then rises to 0.895, while validation accuracy stays at 55–58%. The model starts to overfit the training period after a few epochs.
- **Consequence:** training to 50 epochs without callbacks would only fit the winter data more closely. This motivates **EarlyStopping with restore_best_weights=True**, which keeps the weights of the best validation epoch.

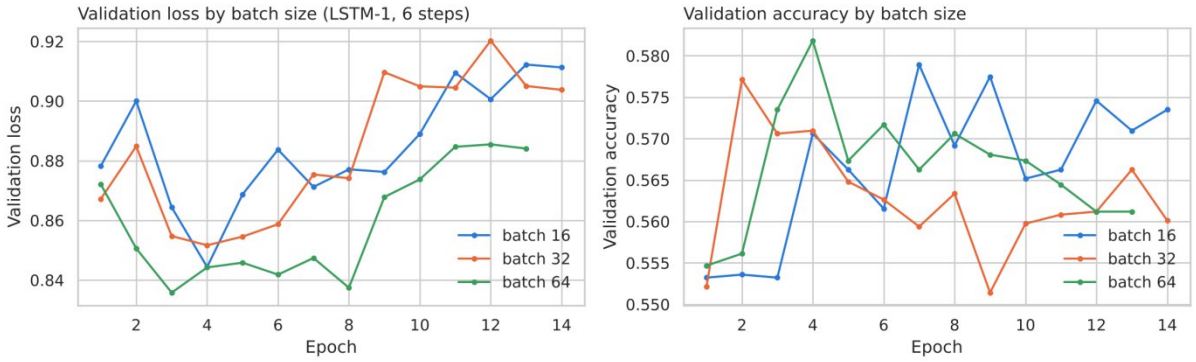


Figure 5: Validation loss and accuracy for batch sizes 16, 32 and 64 (LSTM-1).

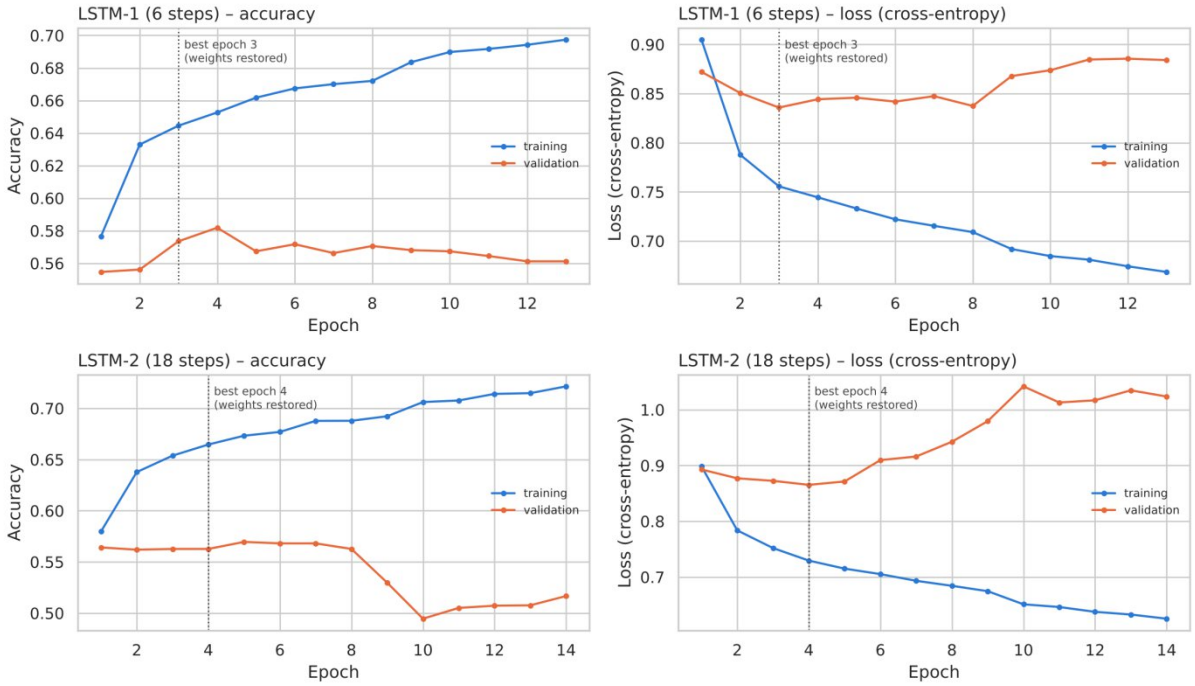


Figure 6: Training vs validation accuracy and loss of LSTM-1 and LSTM-2 (batch size 64).

- **Batch size:**

Table 5: Batch-size comparison (LSTM-1, 6 steps).

Batch	Best validation loss	Training time
16	0.844	85 s (6.1 s/epoch)
32	0.852	59 s (4.2 s/epoch)
64	**0.836**	**29 s (2.3 s/epoch)**

Batch size 64 is chosen: it has the lowest validation loss and trains 2.9× faster than batch 16. The validation accuracy is almost the same for all three (57.1–57.4%), so smaller, noisier batches brought no benefit on this noisy target.

- **Callbacks:** every run reached its best validation loss at **epoch 3–4** and was stopped by EarlyStopping after 13–14 epochs (patience 10), with the best weights restored. ReduceLROnPlateau halved the learning rate once (0.001 → 0.0005), but the validation loss did not improve again. The limit is the difference between the training and validation periods, not the optimisation.
- **Training curves:** training accuracy keeps rising (to about 70%) while validation accuracy peaks at about 57–58% and validation loss rises after epoch 3–4. This is the typical overfitting pattern. The validation windows (the last 20% of training, 28 Mar – 16 Apr) are later and warmer than most of the training data, so what the model learns about winter only partly transfers.
- **LSTM-2 (18 steps) overfits faster:** its validation loss climbs from 0.865 to 1.04 and its validation accuracy drops to about 50% by epoch 10. It also takes 1.9× longer to train (55 s vs 29 s).

3.4 Evaluation

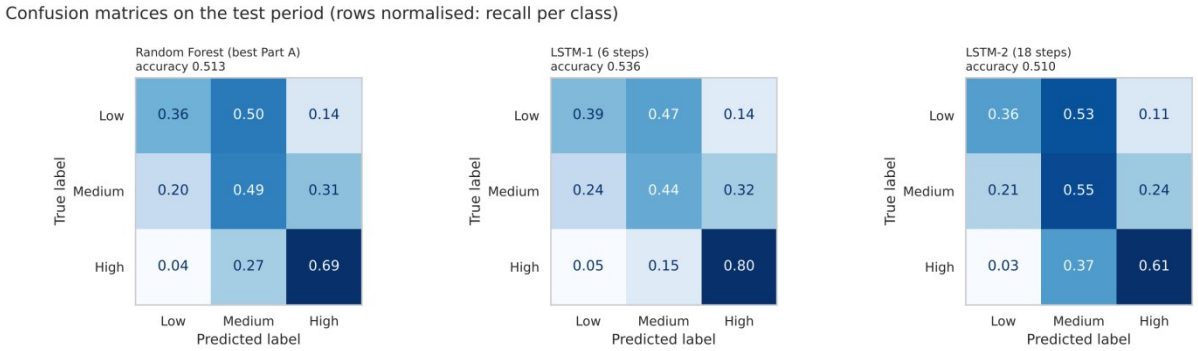


Figure 7: Confusion matrices on the test period (recall per class).

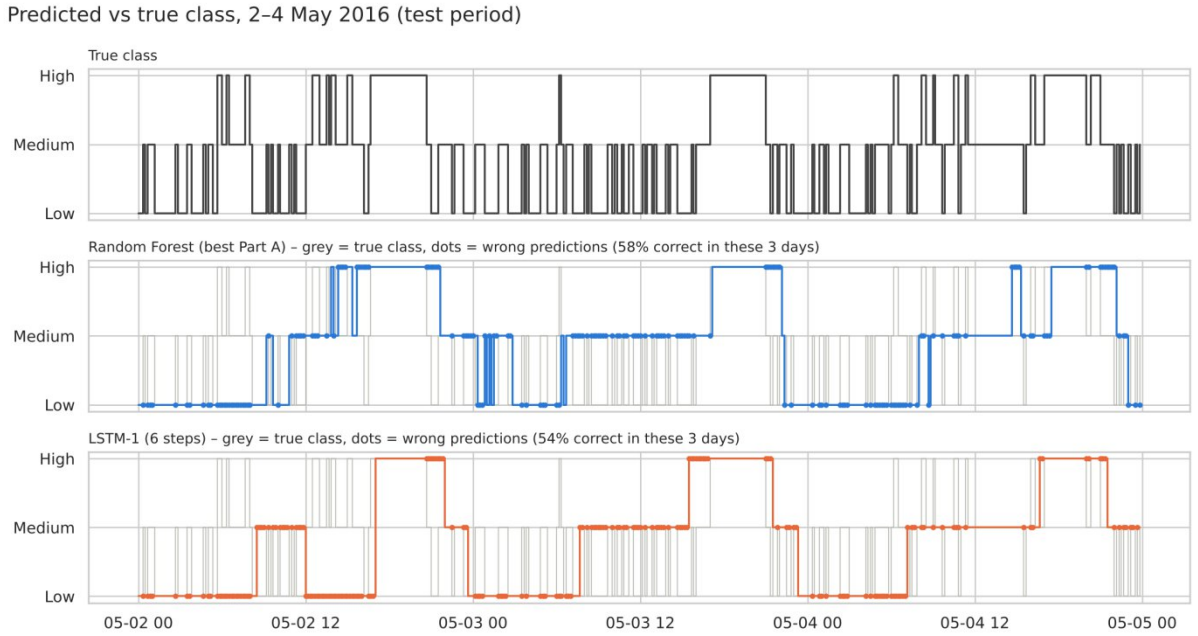


Figure 8: True and predicted class over three days of the test period.

Yes – the 1-hour LSTM (LSTM-1) is the best model, but only by a small margin.

- **Results on the same 5,921 test readings:**

Table 6: LSTM vs best traditional model on the same test readings.

Model	Accuracy	Loss	Weighted F1
LSTM-1	**53.61%**	**0.883**	**0.524**
Random Forest (best Part A)	51.28%	0.917	0.506
LSTM-2	50.95%	0.913	0.506

LSTM-1 gains **+2.3 pp** accuracy and has a lower loss than the Random Forest. LSTM-2 only matches it.

- **Where the gain comes from:** LSTM-1 recognises **High** readings much better (recall **0.80** vs 0.69). A high-consumption event such as cooking builds up over the previous hour (lights on, kitchen temperature and humidity rising), and only a model that sees the sequence can use this build-up. On Low (0.39 vs 0.36) and Medium (0.44 vs 0.49) the two models are similar.

- **Is the difference real?** Over 4 random seeds:

Table 7: Test accuracy over 4 random seeds.

Model	Mean ± std	Range
LSTM-1	**53.19% ± 1.64**	50.8–54.5%
Random Forest	52.08% ± 0.69	51.3–52.9%
LSTM-2	51.59% ± 1.23	–
MLP	50.55% ± 2.29	–

LSTM-1 is ahead on average (+1.1 pp), but the ranges overlap and the LSTM varies more between training runs. The advantage is consistent but modest.

- **Why the gain is not larger:**
  - **Seasonal shift.** The shift between the winter training data and the spring test data limits every model to 51–54%.
  - **No past consumption.** The sequences only contain sensor readings. Appliance use is driven by people, and the sensors register it indirectly and with a delay.
  - **Time of day.** Much of the predictable pattern is the daily routine, which the Random Forest already gets from hour_sin and hour_cos.
- **What the predictions look like:** the 3-day figure shows that both models predict smooth blocks that follow the daily routine (Low at night, Medium during the day, High in the evening). The true class switches every 20–70 minutes. The LSTM's High blocks match the evening peaks better, but neither model captures the fast switching.

3.5 Saved models

Table 8: Files saved in models/ (a reloaded .h5 model gives identical predictions).

File	Content	Size
lstm_1_seq6.h5	LSTM-1 (6 steps)	0.48 MB
lstm_2_seq18.h5	LSTM-2 (18 steps)	0.48 MB
lstm_1_seq6_history.pkl, lstm_2_seq18_history.pkl	training histories	< 0.01 MB
random_forest.pkl, mlp_classifier.pkl	Part A models	13.65 MB, 0.19 MB
standard_scaler.pkl, minmax_scaler.pkl	fitted scalers	< 0.01 MB

4. Part C – Final Summary and Reflection

Table 9: Master results table (same 5,921 test readings; best in bold).

Model type	Model name	Primary: test accuracy	Secondary: test loss	Weighted F1	Accuracy over 4 seeds
Traditional ML	Random Forest	0.5128	0.9174	0.5055	0.5208 ± 0.0069
Traditional ML	MLP Classifier	0.4849	1.4593	0.4860	0.5055 ± 0.0229
Deep Learning 1	**LSTM-1 (6 steps, batch 64)**	**0.5361**	**0.8831**	**0.5235**	**0.5319 ± 0.0164**
Deep Learning 2	LSTM-2 (18 steps, batch 64)	0.5095	0.9130	0.5062	0.5159 ± 0.0123

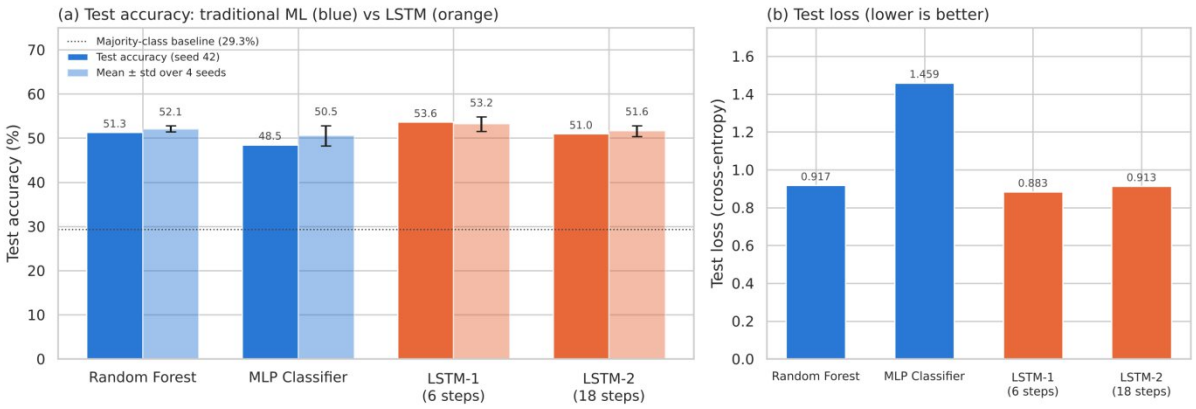


Figure 9: Test accuracy (with the 4-seed mean ± std) and test loss of all models.

4.1 Best model

- **Best model: LSTM-1** (2 stacked LSTM layers, 6-step = 1-hour sequence, batch size 64). It has the highest test accuracy (53.6%), the lowest test loss (0.883), the highest weighted F1 (0.524) and the best mean accuracy over 4 seeds (53.2%).
- **Why it outperformed the others:**
  - It is the only model that sees the recent history of the house, so it can detect the build-up to a high-consumption event (High recall 0.80).
  - The 1-hour window matches the memory of the data (autocorrelation 0.32 at 1 hour, about 0 at 6 hours), while the 3-hour window mainly added noise and overfitted faster.
  - The small architecture with dropout and early stopping limits overfitting on 13,809 training windows.
- **Would I deploy it in production? Not in its current form.**
  - **Accuracy:** at 53.6%, almost one prediction in two is wrong.
  - **Margin over the Random Forest:** the gain is only 1–2 pp. The Random Forest is simpler, trains in about 5 s instead of about 30 s, needs no deep-learning runtime and explains itself through feature importances.
  - **Seasonal drift:** the gap between cross-validation (≈ 58–59%) and the spring test period (≈ 51–54%) shows that performance drops when the season changes. A production system would need at least a full year of data, regular retraining (for example monthly on a sliding window) and monitoring of accuracy over time.
  - **What is in its favour:** all inputs are available in real time from the sensors and a prediction costs almost nothing. It could already support a low-risk use, such as an energy dashboard that flags likely high use, but not automatic control decisions.

4.2 Reflection questions

1. How did the architecture choices affect performance?

- **Sequence length had the largest effect.** 6 steps (1 hour) beat 18 steps (3 hours) by 2.7 pp on seed 42 (53.6% vs 51.0%) and by 1.6 pp on average over 4 seeds (53.2% vs 51.6%), and trained 1.9× faster. The longer window adds readings that carry little

information (autocorrelation 0.17 at 3 hours) and gives the model more ways to overfit: its validation loss rose from 0.87 to 1.04.

- **Batch size mainly affected speed.** 64 gave the lowest validation loss and was 2.9× faster than 16; validation accuracy changed by less than 0.5 pp.
- **Capacity, dropout and early stopping:** the required 64 → 32 LSTM + 16 dense architecture (36,291 parameters) already overfits within 3–4 epochs. Dropout 0.2 and EarlyStopping with restored best weights were essential: without callbacks the validation loss rose from 0.852 to 0.895 within 10 epochs. A bigger network would not help, while a smaller one or stronger dropout might.
- **Learning-rate schedule:** ReduceLROnPlateau halved the learning rate once but did not bring the validation loss down again. The bottleneck is the change in the data over time, not the optimiser.

2. Did the deep learning model outperform the traditional ML models? Why or why not?

Partly. **LSTM-1 outperformed both traditional models**: +2.3 pp accuracy over the Random Forest on seed 42, +1.1 pp on average over 4 seeds, a lower loss and a much better recall of High. It can learn temporal patterns (what happened in the last hour) that a model looking at one reading cannot see. **LSTM-2 did not outperform the Random Forest**, so the benefit depends on choosing the right amount of history. The margin is small for three reasons:

- All models share the same features and suffer from the same winter → spring shift.
- The input does not include past consumption.
- On tabular sensor features, a tuned tree ensemble is a strong baseline that is hard to beat.

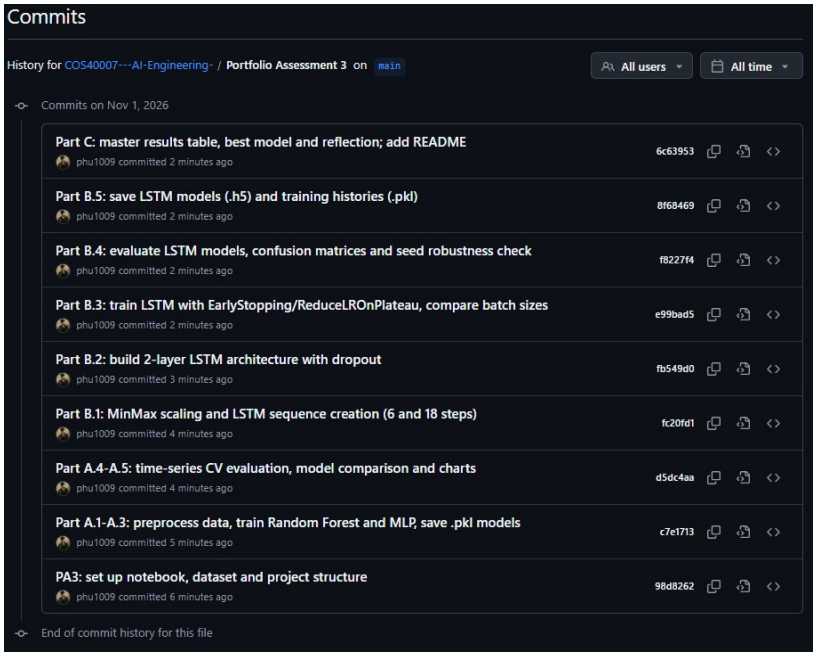
3. If I had more time, what would I do to improve performance further?

- **Add past consumption to the input** (lagged Appliances or its class). Its autocorrelation is 0.75 at 10 minutes, so this is likely to be the largest single gain. The task would then become "predict the next 10 minutes".
- **Handle the seasonal drift:** collect a full year of data, train on a sliding window and retrain regularly, and validate with walk-forward (expanding-window) splits instead of one split.
- **Tune the LSTM with walk-forward validation:** sequence lengths of 3–12 steps, the number of units, dropout and learning rate. Also try a GRU, a 1D-CNN + LSTM, or attention.
- **Compare a stronger tabular baseline:** gradient boosting (LightGBM or XGBoost) with lag and rolling-window features.
- **Average several LSTM seeds (an ensemble)** to remove the ±1.6 pp run-to-run variation.

5. Part D – GitHub Repository

Repository: https://github.com/phu1009/COS40007---AI-Engineering-

The work was committed part by part (set-up, Part A, Part B, Part C and report).



6. Part E – GenAI Reflection

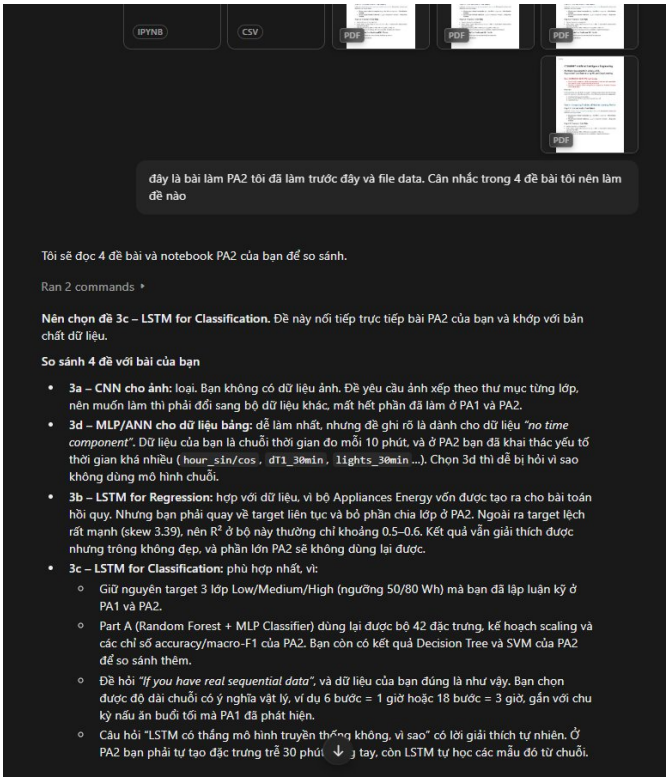
1. GenAI Tools Used

- Claude (Anthropic): choosing option 3c, analysing the instructions and rubric and suggest the plan to do the Assignment, and organising and formatting this report.

2. Specific Tasks & Prompts

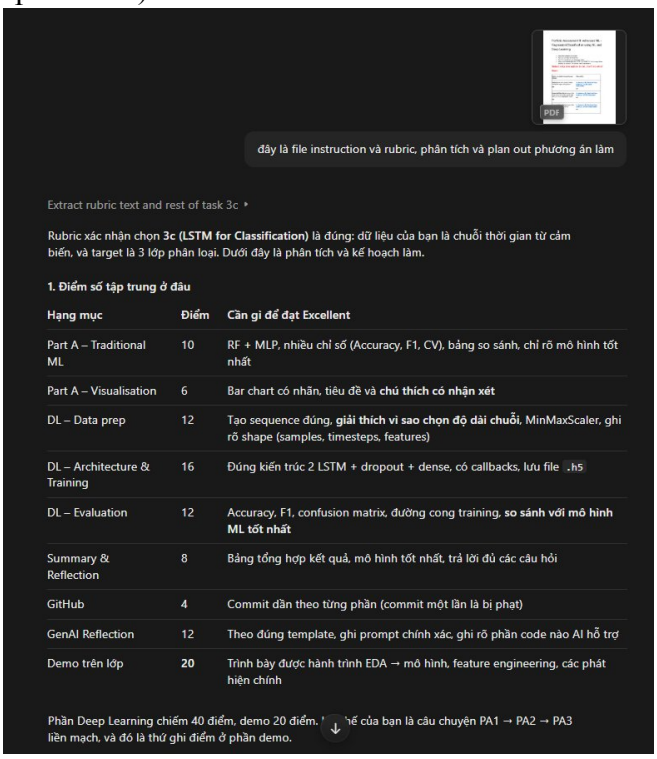
Task 1: Analysing the work on the PA2 and suggesting which to choose among four topic options.

Prompt used: "đây là bài làm PA2 tôi đã làm trước đây và file data. Cân nhắc trong 4 đề bài tôi nên làm đề nào"
(translation: "Here is the PA2 assignment I completed previously, along with the data file. Given the four available topics, which one do you think I should choose?")



- Task 2: Analysing the instructions and rubric and suggest the plan to do the Assignment

  Prompt used: "đây là file instruction và rubric, phân tích và plan out phương án làm" (translation: "Here are the instructions and the rubric, analyze them and map out a plan for how to proceed.")



3. Learning Reflection

- How did GenAI help me learn:

  Claude worked mostly as a planning partner. When I gave it my PA2 work and the dataset, it helped me see why option 3c (LSTM for Classification) made sense: my data is a real time series and my target was already turned into classes in PA2. Having the rubric broken down into a step-by-step plan also meant I knew early on what each part needed, so I spent more of my time on the experiments and less on working out what was being asked. Whenever I didn't fully understand an idea, for example why *return_sequences=True* is needed on the first LSTM layer, or why a shuffled validation split looks too good on time-series data, I could ask follow-up questions until it made sense. Then I checked it against my own results.

- What challenges did it create?

  The biggest challenge was not relying on it too much. The plan it suggested was clear and sounded convincing, so it was easy to just follow it without asking whether each choice really fit my data. For example, I didn't want to pick the sequence length just because it was suggested. I picked it from the autocorrelation and class-persistence analysis (Table 3), so I could actually justify the 6-step and 18-step windows. Another challenge was that GenAI explanations are sometimes very general. Things like "LSTMs are better at temporal patterns" didn't hold up neatly in my results, since LSTM-2 did worse than the Random Forest. I had to explain that difference myself.

- What specific skills did I develop?

- Preparing time-series data properly: splitting chronologically, fitting the scalers only on the training period, and building sliding windows so the LSTM is tested on exactly the same 5,921 readings as the Part A models.

- Reading training curves to spot overfitting, and using EarlyStopping and ReduceLROnPlateau on purpose rather than by default.

- Working out LSTM parameter counts by hand and understanding why sequence length doesn't change the parameter count.

- Comparing models fairly, using the same test set and averaging over several seeds instead of trusting a single run.

- Writing better prompts: giving the tool my own files and the rubric got much more useful answers than general questions.

- Limitations and ethical considerations:

  Claude can't see my notebook outputs unless I give them to it, so anything it says about results has to be checked against what my code actually produced. All the numbers, figures and conclusions in this report come from my own runs. GenAI can also sound confident when it's wrong, so I treated its suggestions as a starting point,

not as the answer. Ethically, I've kept the decisions and the analysis my own and used GenAI for planning, explanations, and organising and formatting the report, and I've disclosed that use openly in this section, as Swinburne's academic integrity guidelines require. I also only shared my own coursework files and the public UCI dataset with the tool, never anything private or belonging to other students.

References

Candanedo, L. (2017). Appliances Energy Prediction [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5VC8G

Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. Energy and Buildings, 140, 81–97. https://doi.org/10.1016/j.enbuild.2017.01.083

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Swinburne University of Technology. (n.d.). Acknowledgement of Country. https://www.swinburne.edu.au/about/our-university/indigenous-matters/acknowledgement/